# Can Taproot-Based Commitments Enable Scalable and Revocable Document Verification in Distributed Communication Networks?

Juan Crespo[a], Sultan Almuhammadi[a], Moatsum Alawida[b], Farag Azzedin[a]

[a]*Information and Computer Science Department, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia*
[b]*Department of Computer Science, Abu Dhabi University, Abu Dhabi, 59911, United Arab Emirates*

---

**Abstract**

Secure verification of digital documents across distributed communication networks requires mechanisms that ensure integrity, authenticity, and efficient revocation while minimizing reliance on centralized verification services. Traditional document verification infrastructures depend on trusted intermediaries and continuous online availability, which introduces scalability constraints and trust limitations in large-scale environments. This paper presents a communication-security architecture for distributed document verification based on a Taproot-enabled commitment mechanism anchored in a single Pay-to-Taproot (P2TR) transaction. The proposed framework aggregates document hashes using a Merkle tree structure and embeds the resulting commitment within Taproot, enabling efficient and scalable verification across distributed systems. Issuer authenticity is ensured through Schnorr signatures that are cryptographically bound to the commitment, while revocation management is implemented through a transaction chaining mechanism that separates immutable issuance commitments from dynamic revocation states. This design enables lightweight verification, where verifiers exchange compact proofs instead of entire document datasets. We provide a formal security analysis under standard cryptographic assumptions and evaluate both communication and verification overhead as functions of the batch size. The results show that Merkle proof size and verification complexity grow logarithmically, $\mathcal{O}(\log N)$, with the number of documents $N$, while the anchoring cost remains constant. These findings demonstrate that the proposed architecture offers a scalable and privacy-preserving solu-

tion for secure document verification in distributed communication systems, without requiring protocol modifications or reliance on trusted third-party infrastructure.

*Keywords:*
Bitcoin, Taproot, Merkle Commitments, Schnorr Signatures, Decentralized Document Verification, On-chain Revocation, Cryptographic Protocols.

---

## 1. Introduction

Document fraud, particularly involving academic credentials, represents a significant global challenge [1]. Verification of document authenticity, integrity, and validity is traditionally handled by the issuing organizations themselves, creating a structural dependency on centralized verification infrastructures [2]. Historically, this process has relied on physical mechanisms such as stamps and, more recently, on centralized digital systems maintained by the issuer. Even in digital form, centralized verification systems incur significant operational costs in order to maintain availability and to mitigate denial-of-service attacks.

Bitcoin is a decentralized consensus network that maintains a globally replicated ledger through a Proof-of-Work mechanism [3]. Its large and sustained computational security budget provides strong resistance against history revision and transaction censorship. Because validation is permissionless and globally distributed, Bitcoin offers high availability and long-term immutability guarantees without reliance on centralized infrastructure [4]. These properties make it a robust substrate for anchoring cryptographic commitments that require durability and public verifiability beyond purely monetary use.

Bitcoin protocol stores only transaction-related data by design, including: amounts, timestamps, and input and output scripts. Nevertheless, several approaches have been proposed to anchor external cryptographic commitments in the Bitcoin blockchain in order to support applications such as digital document verification. One widely used technique relies on the `OP_RETURN` opcode, which allows a limited amount of arbitrary data to be embedded directly in a transaction output. Another approach is OpenTimestamps, an open timestamping protocol in which document hashes are aggregated into Merkle trees and periodically anchored in the Bitcoin blockchain by calendar servers operated by third parties. In this model, the Merkle tree construc-

tion and anchoring transaction are performed externally, and the associated transaction fee is paid by the calendar operator, leaving users without direct control over how batches are formed or when anchoring occurs. Blockcerts represents a higher-level framework for issuing and verifying digital credentials, where document hashes are anchored on-chain while certificate metadata is managed off-chain. While this framework enables verifiable credentials, its primary objective is credential representation and management rather than the design of cryptographic commitment structures for decentralized verification.

Despite their usefulness, these approaches remain limited in at least one of three important dimensions: scalable batch aggregation, privacy-preserving per-document verification, or explicit and auditable revocation support. In particular, OP_RETURN-based techniques expose commitment data publicly and do not provide native mechanisms for issuer authentication or document life-cycle management. OpenTimestamps enables scalable integrity anchoring but does not address issuer authentication or revocation, while Blockcerts focuses primarily on credential representation rather than on commitment structures designed for decentralized verification.

Recent upgrades to the Bitcoin protocol introduce new capabilities that may help address these limitations. In particular, Taproot is a protocol upgrade that combines Schnorr signatures and Merkleized Abstract Syntax Trees (MAST) to enable more efficient and privacy-preserving representation of complex spending conditions on the Bitcoin blockchain. Taproot introduces a new output type known as Pay-to-Taproot (P2TR), in which coins are locked to a Taproot output key that commits to both a public key and optional script paths encoded through MAST.

The central question addressed in this work is whether Taproot is sufficient to implement a decentralized document verification protocol satisfying scalability, privacy, revocation support, and formally stated security guarantees on Bitcoin.

More precisely, we investigate whether a Taproot-native construction can simultaneously achieve the following:

1. Scalable batch aggregation of document commitments,
2. Privacy-preserving per-document verification,
3. Explicit and auditable revocation support,
4. Cryptographic security grounded in formally stated assumptions.

The challenge is to design a construction that satisfies these properties while preserving indistinguishability from standard P2TR outputs and remaining fully compatible with the Bitcoin protocol.

To this end, we introduce a Taproot-native commitment construction that enables batch document verification using a single P2TR transaction. The proposed design leverages non-executed Taproot script paths to encode cryptographic commitments, combined with MAST and Schnorr signatures standardized in BIPs [5, 6]. Verification is performed entirely off-chain through reconstruction of the Taproot output key and validation of Merkle authentication paths and Schnorr signatures using a proof-of-validation object distributed with each document.

To the best of our knowledge, this work provides one of the first systematic designs of a Taproot-native protocol for decentralized document verification on Bitcoin.

The main contributions of this work are as follows:

1. We formalize a Taproot-based commitment structure that embeds document hashes and issuer public keys within non-executed script paths while preserving indistinguishability from standard P2TR outputs.
2. We design a batch document verification protocol that enables per-document integrity and issuer authenticity verification using Merkle authentication paths and Schnorr signatures.
3. We propose a revocation mechanism based on transaction chaining that updates the revocation state without modifying or invalidating the original issuance commitment.
4. We provide an analytical characterization of on-chain transaction cost and off-chain verification complexity as functions of batch size, showing logarithmic $\mathcal{O}(\log N)$ verification complexity and constant anchoring cost.
5. We define a formal security model and provide reduction-based arguments establishing integrity under the collision resistance of the hash function and authenticity under the existential unforgeability under chosen-message attack (EUF-CMA) of Schnorr signatures, assuming the immutability of the Bitcoin blockchain.

## 2. Related Work

This section reviews prior work on information anchoring techniques in the Bitcoin blockchain and on the use of smart contracts in Ethereum, with

a focus on their application to document verification systems and their limitations in terms of scalability, privacy, and revocation support.

### *2.1. Information Anchoring Techniques in Bitcoin*

The Bitcoin protocol adopts a deliberately restrictive execution model based on the Unspent Transaction Output (UTXO) abstraction and a non-Turing-complete scripting language. Scripts have no access to persistent state, arbitrary function calls, or iterative control structures, which constrains the design space for data anchoring and commitment mechanisms. These architectural restrictions directly impact how document verification systems can be implemented at the base layer.

Early data anchoring approaches encoded arbitrary bytes into payment addresses, effectively embedding metadata within transaction outputs [7]. These methods incur linear on-chain growth with batch size, permanently lock bitcoin in unspendable outputs, and introduce unnecessary UTXO set expansion. As a result, fake address-based schemes are economically inefficient and do not scale for structured document verification systems.

With the introduction of additional payment script types, alternative data anchoring techniques have been proposed. Comprehensive analyses classify these approaches into two main categories: script-based methods, which increase construction and verification complexity, and fake address-based methods, which incur higher cost and lack privacy [8]. Among these, OP_RETURN-based techniques explicitly mark outputs as unspendable and avoid UTXO set growth, thereby reducing negative impact on the network [9]. While OP_RETURN has historically been constrained by policy limits, it remains the most widely adopted mechanism for anchoring non-monetary data in Bitcoin.

The introduction of Segregated Witness (SegWit) reorganized transaction structure by isolating witness data and applying discounted weight to signature material, reducing effective transaction cost [10]. This enabled lower-cost commitment schemes that leverage witness fields for data anchoring. However, SegWit-based techniques remain limited to integrity anchoring and do not natively support issuer authentication, explicit revocation, or structured batch verification within a unified design.

Several standardized solutions are widely deployed on Bitcoin today. Blockcerts [11] and OpenTimestamps [12] are representative Bitcoin-based systems for credential anchoring and timestamping, respectively. However, both remain primarily integrity-oriented and do not provide a Taproot-native mech-

Table 1: Functional comparison of existing Bitcoin anchoring approaches and the proposed Taproot-based protocol.

| **Property** | **Fake Address** | **OP_RETURN** | **Script** | **Witness** | **OpenTimestamps** (OTS) | **Proposed** (Taproot) |
|---|---|---|---|---|---|---|
| Integrity Verification | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Issuer Authentication | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| Document Revocation | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| Batch Support | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| Privacy | ✗ | LOW | HIGH | HIGH | MEDIUM | HIGH |

anism that cryptographically couples issuer authenticity and revocation to the anchoring output itself.

Table 1 summarizes the functional properties supported by representative Bitcoin-based document anchoring techniques. The comparison focuses on key capabilities relevant for document verification systems, including batch commitments, issuer authentication, revocation support, privacy preservation, and independent verification. Early techniques such as Fake Address and OP_RETURN primarily provide document integrity by anchoring hashes directly on-chain, but they lack mechanisms for issuer authentication and lifecycle management. Script and witness-based constructions improve flexibility but remain integrity-oriented. OpenTimestamps introduces scalable batch commitments through Merkle aggregation; however, it does not natively provide issuer authentication or explicit revocation mechanisms. In contrast, the proposed Taproot-based construction integrates batch commitments, issuer authenticity, and revocation support within a unified protocol design.

Table 2 compares the computational and on-chain efficiency of different Bitcoin anchoring approaches. The evaluation considers transaction footprint, batching efficiency, and verification overhead. Legacy techniques such as Fake Address and OP_RETURN incur linear on-chain growth because each document requires an individual commitment. Script and witness-based approaches improve fee efficiency but still scale linearly with the number of documents. OpenTimestamps reduces the on-chain footprint by aggregating document hashes into a Merkle tree, enabling constant anchoring cost and logarithmic verification proofs. The proposed Taproot-based construction preserves these scalability properties while integrating issuer authentication

Table 2: Network impact and transaction cost characteristics of Bitcoin-based document anchoring approaches.

| **Property** | **Fake Address** | **OP_RETURN** | **Script** | **Witness** | **OpenTimestamps** (OTS) | **Proposed** (Taproot) |
|---|---|---|---|---|---|---|
| On-chain footprint | HIGH | LOW | HIGH | MEDIUM | LOW | LOW |
| UTXO impact | HIGH | NONE | LOW | LOW | LOW | LOW |
| TX Cost Model (BTC) | Burn + fee | Standard fee | Standard fee | Discounted SegWit | Shared anchoring | Discounted SegWit |
| Verification efficiency | LOW | LOW | LOW | LOW | LOW | HIGH |
| Size (bytes) | $20 \times N$ | $\sim 80$ | $\sim 1000$ | $\sim 4000$ | $\sim 40$–$80$ | $\sim 32$ |

and revocation support within the same commitment structure.

Due to Bitcoin's intentionally restricted execution model, many document verification systems have migrated toward more expressive blockchain platforms. Ethereum introduces an account-based state model and a Turing-complete virtual machine that enables persistent storage and arbitrary contract logic [13]. This architectural difference allows on-chain verification and revocation logic to be implemented directly within smart contracts, but introduces higher execution complexity, gas-based cost variability, and a broader attack surface. Table 3 summarizes the architectural and security differences between Bitcoin and Ethereum in the context of document verification systems.

Ethereum enables native on-chain verification through Turing-complete smart contracts and persistent global state, but this flexibility introduces gas-based execution costs, increased systemic complexity, and a broader attack surface. In contrast, Bitcoin relies on a UTXO model with restricted, non-Turing-complete Script, where verification logic is performed off-chain using cryptographic proofs anchored by on-chain commitments. Although this model limits computational expressiveness, it reduces execution complexity and minimizes the base-layer attack surface. For this reason, this work adopts Bitcoin as the underlying platform, favoring a minimal execution model, predictable fee structure, and strong energy-based security guarantees over contract-dependent on-chain verification.

Numerous studies leverage Ethereum smart contracts to implement certification and attestation systems that require expressive verification logic [14, 15, 16, 17, 18, 19]. While these approaches demonstrate the flexibility of Ethereum's execution model, they incur increased on-chain execution over-

Table 3: Condensed architectural and security comparison between Bitcoin and Ethereum for document verification systems.

| **Dimension** | Bitcoin (PoW, UTXO) | Ethereum (PoS, Account/VM) |
|---|---|---|
| **Security basis** | Energy-based security (hashrate $H$) | Stake-based security (total stake $S$) |
| **Adversarial threshold** | $\alpha = H_A/H_{total}$ | $\alpha = S_A/S_{total}$ |
| **State model** | UTXO; stateless validation | Global account state; persistent storage |
| **Execution model** | Limited, non-Turing Script | Turing-complete EVM |
| **Verification approach** | On-chain commitment, off-chain proof verification | On-chain contract-based verification |
| **Cost model** | Fee per transaction size (vbytes) | Gas-based fee (computation + storage) |
| **Revocation mechanism** | New commitment transaction required | Contract state update |
| **Finality model** | Probabilistic finality | Economic finality |
| **Attack surface** | Minimal scripting layer | VM and smart-contract dependent |
| **Governance style** | Conservative soft-forks | Frequent hard-fork upgrades |

head, higher gas fees, and greater implementation complexity.

Institutional adoption of blockchain-based document verification has also been studied extensively. Castro et al. [20] analyze blockchain adoption for higher education diplomas. Rustemi et al. [21] show that most academic certificate verification platforms are implemented on Ethereum and identify document revocation as a major unresolved challenge. Recent research further highlights unresolved challenges related to privacy and revocation in decentralized credential systems [22, 23].

### *2.2. Limitations of Existing Approaches and Formal Gap*

Existing Bitcoin-based anchoring mechanisms mainly provide *data integrity* guarantees. However, most of these approaches do not simultaneously support issuer authentication, scalable batch commitments, and explicit revocation within a unified cryptographic framework. Early techniques based on fake addresses and `OP_RETURN` embed data directly into Bitcoin transactions. While these approaches are simple to implement, they suffer from practical limitations. In particular, fake-address techniques destroy spendable value,

while `OP_RETURN`-based methods expose metadata explicitly on-chain, which may reduce privacy and increase blockchain footprint.

OpenTimestamps improves scalability by aggregating document hashes using Merkle trees before anchoring them to the blockchain. Although this approach significantly reduces the number of required transactions, it does not cryptographically bind the commitment to a specific issuer. Moreover, OpenTimestamps does not provide a native mechanism for document revocation. Another limitation is that the Merkle aggregation process is typically managed by an external service, which reduces issuer-level control over the commitment structure.

More recent approaches use script- or witness-based constructions, especially after the activation of SegWit. These techniques improve fee efficiency and reduce UTXO growth by placing commitment data in the witness field. Despite these improvements, these solutions remain primarily integrity-oriented and do not provide mechanisms for issuer authentication or lifecycle management of anchored documents.

Ethereum-based certification systems attempt to overcome these limitations by using smart contracts to implement verification and revocation logic directly on-chain. This enables dynamic state updates and explicit revocation tracking. However, this flexibility comes with several trade-offs, including higher execution complexity, gas-based cost variability, and a broader attack surface due to the use of Turing-complete contract execution. Furthermore, these systems rely on different security assumptions, including contract correctness and economic finality mechanisms.

From a formal perspective, current approaches do not simultaneously achieve the following properties within a purely Bitcoin-native construction:

- $\mathcal{O}(1)$ on-chain batch commitment cost,
- $\mathcal{O}(\log N)$ per-document verification proof,
- cryptographic binding of issuer identity to the on-chain commitment,
- explicit and publicly auditable revocation state,
- complete off-chain verification without requiring expressive on-chain execution.

We consider an adversary capable of: (i) forging a document that was not included in the original commitment, (ii) presenting a revoked document as

valid, or (iii) modifying validation proofs. The adversary is assumed not to control a majority of the total network hash rate at the time the commitment transaction is confirmed.

In contrast to smart-contract-based approaches, the construction proposed in this work relies exclusively on standardized Bitcoin primitives and off-chain verification. This design avoids on-chain execution overhead while preserving cryptographic verifiability and publicly auditable revocation state transitions. To the best of our knowledge, no prior work systematically leverages Taproot script-path commitments as non-executed commitment carriers within a complete decentralized document verification protocol that explicitly separates issuance and revocation state. The proposed Taproot-based protocol mitigates these limitations by anchoring an issuer-defined Merkle structure within a single P2TR output, achieving $\mathcal{O}(1)$ on-chain batch issuance while maintaining $\mathcal{O}(\log N)$ verification complexity off-chain. Unlike OP_RETURN or OTS-based methods, it supports native issuer authentication through Schnorr signatures bound to the committed structure and enables explicit revocation via controlled state updates. This design preserves Bitcoin's minimal trust model while extending its functional capability, achieving scalability, privacy, and semantic completeness without introducing the systemic complexity of general-purpose smart contracts.

## 3. Background

Bitcoin transactions are validated using a script-based execution model that specifies the conditions under which transaction outputs may be spent. The Bitcoin protocol follows the UTXO model, where each transaction consumes previously created outputs and generates new ones. Spending conditions are encoded using Bitcoin Script, a stack-based and intentionally non-Turing-complete language designed to guarantee predictable execution complexity and minimize the network's attack surface.

Bitcoin Script does not support persistent state, loops, or arbitrary function calls. These restrictions enforce deterministic validation and enable decentralized verification even by resource-constrained nodes. Consequently, protocol constructions built on Bitcoin must rely on cryptographic commitments and off-chain verification logic rather than on expressive on-chain stateful computation.

### *3.1. Schnorr Signatures in Bitcoin*

With the activation of BIP340, Bitcoin adopted Schnorr digital signatures as the signature scheme for Taproot outputs. Let $G$ denote the generator of the secp256k1 elliptic curve group of prime order $n$, and let $x \in \mathbb{Z}_n$ denote a private scalar with corresponding public key $P = xG$.

A Schnorr signature on a message $m$ consists of a pair $(R, s)$, where $R = kG$ for a nonce $k \in \mathbb{Z}_n$ and

$$s = k + H(R \parallel P \parallel m)\, x \pmod{n},$$

where $H(\cdot)$ denotes a cryptographic hash function. The exact signature encoding and message hashing procedure follow the Schnorr signature construction standardized in BIP340.

Schnorr signatures provide linearity with respect to group operations, enabling public key aggregation and multi-party signature constructions under compatible message models. These properties reduce verification overhead and transaction size while maintaining provable security under standard cryptographic assumptions. In addition, Schnorr signatures eliminate the signature malleability issues present in earlier schemes, enabling more robust protocol constructions.

### *3.2. Merkleized Abstract Syntax Trees (MAST)*

MAST allows multiple alternative spending conditions to be committed within a single transaction output. Each spending condition is encoded as an individual script, and the hashes of these scripts form the leaves of a Merkle tree.

Let $\{S_i\}$ denote a set of scripts and let $\ell_i = H_{\text{TapLeaf}}(S_i)$ be their corresponding TapLeaf hashes. The Merkle root $M$ is computed as

$$M = \mathsf{TapMerkleRoot}(\ell_1, \ldots, \ell_N).$$

When spending an output via a script path, only the executed script $S_i$ and the corresponding Merkle authentication data are revealed on-chain. Unused scripts remain hidden, improving privacy and reducing information disclosure. In the context of this work, however, script-path spending is not required, and MAST is used exclusively as a commitment mechanism without revealing any script data on-chain. MAST therefore enables compact commitments to multiple conditions without revealing unused branches.

### *3.3. Taproot and Pay-to-Taproot Outputs*

Taproot, introduced through BIP341, integrates Schnorr signatures and MAST into a unified output type known as P2TR. A P2TR output commits to an internal public key $P_{\text{int}}$ and an optional Merkle root $M$ representing alternative script paths.

The Taproot output key $P_{\text{out}}$ is computed using a key-tweaking mechanism defined as

$$P_{\text{out}} = P_{\text{int}} + H_{\text{TapTweak}}(P_{\text{int}} \parallel M)\, G,$$

where $H_{\text{TapTweak}}(\cdot)$ is the tagged hash function defined in BIP341. If no script paths are present, $M$ is set to the empty value.

Spending a P2TR output can occur via one of two paths: the key path or the script path. In the key path, the spender provides a Schnorr signature corresponding to the Taproot output key $P_{\text{out}}$. In the script path, the spender reveals a script $S_i$ together with the corresponding Merkle authentication data and a valid witness satisfying the script conditions.

Because both key-path and script-path spends are represented as Taproot transactions, standard blockchain analysis cannot distinguish whether a transaction encodes a simple payment, a complex spending policy, or an application-specific commitment.

As a result, Taproot preserves transaction indistinguishability even when embedding commitments to complex verification logic or multi-party authorization schemes, without revealing unused conditions or the existence of alternative spending paths.

Since its activation in November 2021, Taproot has been widely adopted across the Bitcoin network, enabling more expressive, efficient, and privacy-preserving transaction constructions. These properties form the cryptographic foundation for higher-level protocols that rely on compact commitments, selective disclosure, and off-chain verification.

## 4. Proposed Protocol

We propose a Taproot-native protocol for scalable and revocable document verification. The system involves four primary entities:

- **Issuer**: the trusted entity responsible for generating and anchoring document commitments on the Bitcoin blockchain.

- **Holder**: the party that possesses a digital document together with its corresponding proof-of-validation.
- **Verifier**: any party that independently verifies the authenticity and validity of a document.
- **Bitcoin Network**: the decentralized infrastructure providing immutable timestamping and transaction validation.

The main abbreviations used throughout the paper are listed in Table 4. All symbols and parameters adopted in the protocol description, security model, and complexity analysis are listed in Table 5. The protocol consists of two main components:

1. The construction of a single P2TR output derived from a batch of $N$ documents and an encapsulated off-chain object referred to as the *proof-of-validation*.
2. An off-chain verification procedure that relies exclusively on the digital document, the proof-of-validation object, and read-only access to a Bitcoin node.

Figure 1 illustrates the interaction model between the issuer, holder, verifier, and the Bitcoin blockchain.

### *4.1. Batch Commitment Construction*

Let us $\{D_i\}_{i=1}^{N}$ denote a batch of $N$ digital documents issued by a single organization. The issuer controls a Schnorr signing key pair $(x_{\text{iss}}, P_{\text{iss}})$ used to guarantee authenticity. For each document $D_i$, the issuer computes:

$$h_i = H(D_i), \tag{1}$$

where $H(\cdot)$ is a collision-resistant hash function (e.g., SHA-256).

*Script-leaf commitments..* Each document is embedded into a Taproot script tree through a dedicated script leaf $S_i$. Concretely, it $S_i$ encodes the tuple $(h_i, P_{\text{iss}}, i)$, where it $i$ binds the commitment to its position within the batch. The corresponding Taproot leaf hash is computed following BIP341 as

$$\ell_i := H_{\text{TapLeaf}}(\mathsf{leaf_version} \parallel \mathsf{len}(S_i) \parallel S_i). \tag{2}$$

Table 4: A list of abbreviations used in the paper.

| **Abbreviation** | **Meaning** |
|---|---|
| BIP | Bitcoin Improvement Proposal |
| BTC | Bitcoin |
| DoS | Denial-of-Service |
| EUF-CMA | Existential Unforgeability under Chosen-Message Attack |
| EVM | Ethereum Virtual Machine |
| MAST | Merkleized Abstract Syntax Trees |
| OTS | OpenTimestamps |
| P2TR | Pay-to-Taproot |
| PPT | Probabilistic Polynomial-Time |
| PoS | Proof-of-Stake |
| PoW | Proof-of-Work |
| SegWit | Segregated Witness |
| UTXO | Unspent Transaction Output |
| vB | Virtual Bytes |

The script is constructed such that script-path execution unconditionally fails. Consequently, spending through the script path is infeasible by construction, and the commitment is intended to be accessed exclusively via the key path. Taproot script paths are therefore used purely as commitment carriers and are never intended to be revealed or executed. Although the script paths are never intended to be executed, using Taproot script leaves provides a structured commitment mechanism compatible with the MAST framework. This design preserves indistinguishability from standard P2TR outputs while allowing future extensibility of script conditions without modifying the on-chain commitment.

*Merkle aggregation..* All leaf hashes $\{\ell_i\}_{i=1}^{N}$ are aggregated according to the Taproot Merkleization rules:

$$M = \mathsf{TapMerkleRoot}(\ell_1, \ell_2, \ldots, \ell_N). \tag{3}$$

The Merkle root $M$ compactly commits to the entire batch of documents.

Figure 2 illustrates the Merkle tree structure used in the proposed construction. Each leaf node corresponds to a Taproot script leaf embedding document-specific data, while internal nodes represent intermediate hashes

Table 5: List of symbols and parameters used in the paper.

| **Symbol** | **Meaning** | **Symbol** | **Meaning** |
|---|---|---|---|
| $a, b$ | Implementation-dependent constants in the verification | $B_k$ | Revocation bitmap published in transaction $T_k$ |
| $C_{\text{iss}}$ | Constant issuance transaction size | $C_{\text{proof}}$ | Constant part of proof size |
| $C_{\text{rev}}$ | Constant overhead of a revocation transaction | $D_i$ | $i^{th}$ document in the batch |
| $D^*$ | Forged candidate document generated by the adversary | $d$ | Depth of the Merkle tree, $\lceil \log_2 N \rceil$ |
| $G$ | Generator point of secp256k1 | $h_i$ | Hash of document $D_i$ |
| $H(\cdot)$ | Collision-resistant hash function | $H_{\text{TapLeaf}}(\cdot)$ | Taproot tagged hash for leaf construction |
| $H_{\text{sig}}(\cdot)$ | Hash used to derive the signed message | $H_{\text{TapTweak}}(\cdot)$ | Taproot tagged hash for output-key tweaking |
| $i$ | Leaf index of a document within the batch | $k$ | Schnorr nonce |
| $\ell_i$ | TapLeaf hash corresponding to script $S_i$ | $M$ | Merkle root committing to the batch |
| $m$ | Generic signed message | $m_i$ | Batch-bound message signed for document $D_i$ |
| $N$ | Number of documents in a batch | $P$ | Public key corresponding to private scalar $x$ |
| $P_{\text{agg}}$ | Aggregated MuSig2 public key | $P_{\text{int}}$ | Internal Taproot public key |
| $P_{\text{iss}}$ | Issuer public signing key | $P_{\text{out}}$ | Tweaked Taproot output key |
| $\pi_i$ | Proof-of-validation associated with document $D_i$ | $\pi^*$ | Forged proof generated by the adversary |
| $\lvert\pi_i\rvert$ | Size of the proof-of-validation for document $D_i$ | $\mathsf{path}_i$ | Merkle authentication path for document $D_i$ |
| $R$ | Nonce commitment in a Schnorr signature | $s$ | Scalar component of a Schnorr signature |
| $S_i$ | Script leaf associated with document $D_i$ | $\sigma_i$ | Schnorr signature for document $D_i$ |
| $T_0$ | Initial anchoring transaction | $T_k$ | $k$-th descendant transaction in revocation chain |
| $t_{\text{ver}}(N)$ | Verification time as a function of $N$ | $\mathsf{txid}_0$ | Anchoring transaction identifier |
| $\rho$ | Mempool fee rate in sat/vB | $\text{vB}(T_0)$ | Size of issuance transaction $T_0$ in virtual bytes |
| $\text{vB}(T_k)$ | Size of revocation transaction $T_k$ in virtual bytes | $\lvert B_k\rvert$ | Size of the revocation bitmap |
| $x$ | Private scalar in Schnorr signatures | $x_{\text{iss}}$ | Issuer private signing key |
| $\mathcal{A}$ | Adversary in the security model | | |

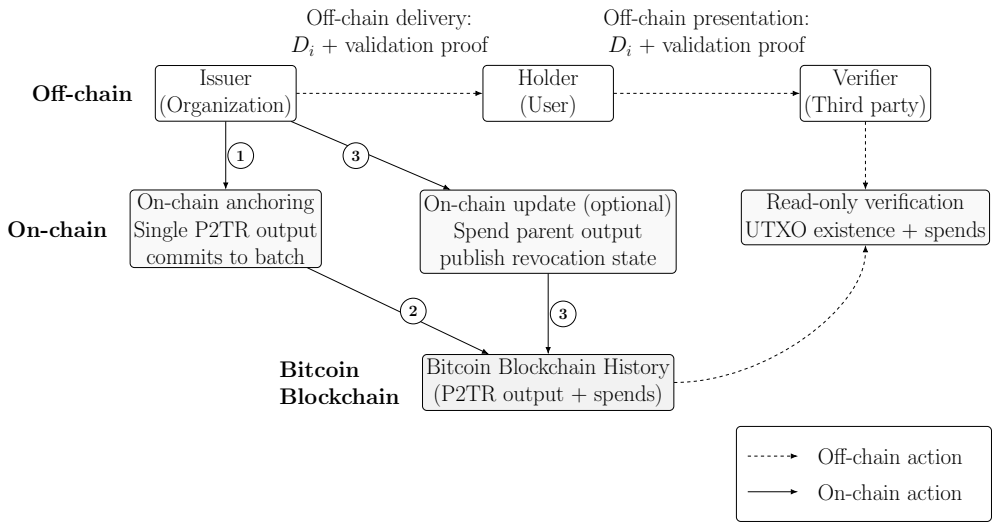


Figure 1: System architecture and interaction flow of the proposed batch document verification protocol. (1) The issuer generates a batch commitment over a set of documents and anchors it on-chain by creating a single P2TR output. (2) Once confirmed, the commitment becomes part of the Bitcoin blockchain history. (3) Optionally, the issuer may later spend the committed output to publish an updated state, enabling revocation tracking. Off-chain, the issuer delivers each document $D_i$ together with its validation proof to the holder, who may later present them to a verifier. Verification is performed independently by checking the commitment and corresponding transactions in the Bitcoin blockchain, including UTXO existence and spending history, without requiring interaction with the issuer.

computed according to the Taproot Merkleization rules. The root $M$ enables efficient per-document verification using a logarithmic-size Merkle authentication path without revealing information about other batch elements.

*Taproot output construction..* Let $P_{\text{int}}$ denote the internal Taproot public key controlled by the issuer. The Pay-to-Taproot output key is derived using the Taproot key-tweaking mechanism:

$$P_{\text{out}} = P_{\text{int}} + H_{\text{TapTweak}}(P_{\text{int}} \parallel M)\, G, \tag{4}$$

where $H_{\text{TapTweak}}(\cdot)$ is the tagged hash defined in BIP341 and $\parallel$ denotes concatenation. The resulting P2TR output $P_{\text{out}}$ commits to $M$ while remaining indistinguishable from a standard key-path Taproot output.

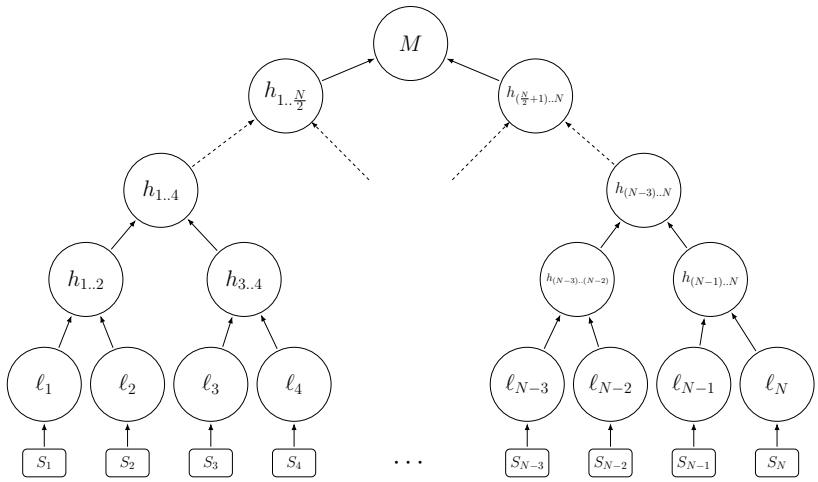


Figure 2: Merkle tree structure aggregating script-leaf commitments for a batch of $N$ documents.

*On-chain anchoring..* The issuer broadcasts a Bitcoin transaction $T_0$ that creates an output paying to $P_{\text{out}}$. Once confirmed, the transaction identifier $\mathsf{txid}_0$ serves as the immutable reference for the batch commitment.

*4.2. Proof Generation*

For each document $D_i$, the issuer generates a proof-of-validation object:

$$\pi_i := (\mathsf{txid}_0,\, i,\, P_{\text{iss}},\, \sigma_i,\, \mathsf{path}_i), \tag{5}$$

where:

- $\mathsf{txid}_0$ is the anchoring transaction identifier,
- $i$ is the leaf index,
- $P_{\text{iss}}$ is the issuer's Schnorr public key,
- $\sigma_i$ is a Schnorr signature over a batch-bound message,
- $\mathsf{path}_i$ is the Merkle authentication path corresponding to $\ell_i$.

To prevent replay across batches, the issuer signs a message that binds the document to its anchoring transaction:

**Algorithm 1** Batch Commitment and Proof Generation

**Require:** Documents $\{D_i\}_{i=1}^{N}$, issuer key $(x_{\text{iss}}, P_{\text{iss}})$, internal key $P_{\text{int}}$
**Ensure:** $P_{\text{out}}$, proofs $\{\pi_i\}_{i=1}^{N}$

```
for i ← 1 to N do
    h_i ← H(D_i)
    Construct S_i encoding (h_i, P_iss, i)
    ℓ_i ← H_TapLeaf(leaf_version ‖ len(S_i) ‖ S_i)
end for
M ← TapMerkleRoot(ℓ_1, ..., ℓ_N)
t ← H_TapTweak(P_int ‖ M)
P_out ← P_int + tG
Broadcast T_0 creating output to P_out
for i ← 1 to N do
    path_i ← MerklePath(i, {ℓ_j})
    m_i ← H_sig(h_i ‖ i ‖ txid_0)
    σ_i ← Sign(x_iss, m_i)
    π_i ← (txid_0, i, P_iss, σ_i, path_i)
end for
return P_out, {π_i}
```

$$m_i := H_{\text{sig}}(h_i \parallel i \parallel \mathsf{txid}_0), \tag{6}$$

$$\sigma_i = \mathsf{Sign}(x_{\text{iss}}, m_i). \tag{7}$$

Algorithm 1 summarizes the batch commitment and proof generation procedure executed by the issuer.

### *4.3. Verification Procedure*

Verification is performed entirely off-chain using the document $D_i$, the proof $\pi_i$, and read-only access to blockchain history. Algorithm 2 describes the verification procedure executed by the verifier. The process reconstructs the committed Merkle root, derives the Taproot output key, verifies the on-chain commitment, checks the issuer's Schnorr signature, and finally validates the revocation state associated with the document.

Figure 3 illustrates the verification workflow executed by the verifier.

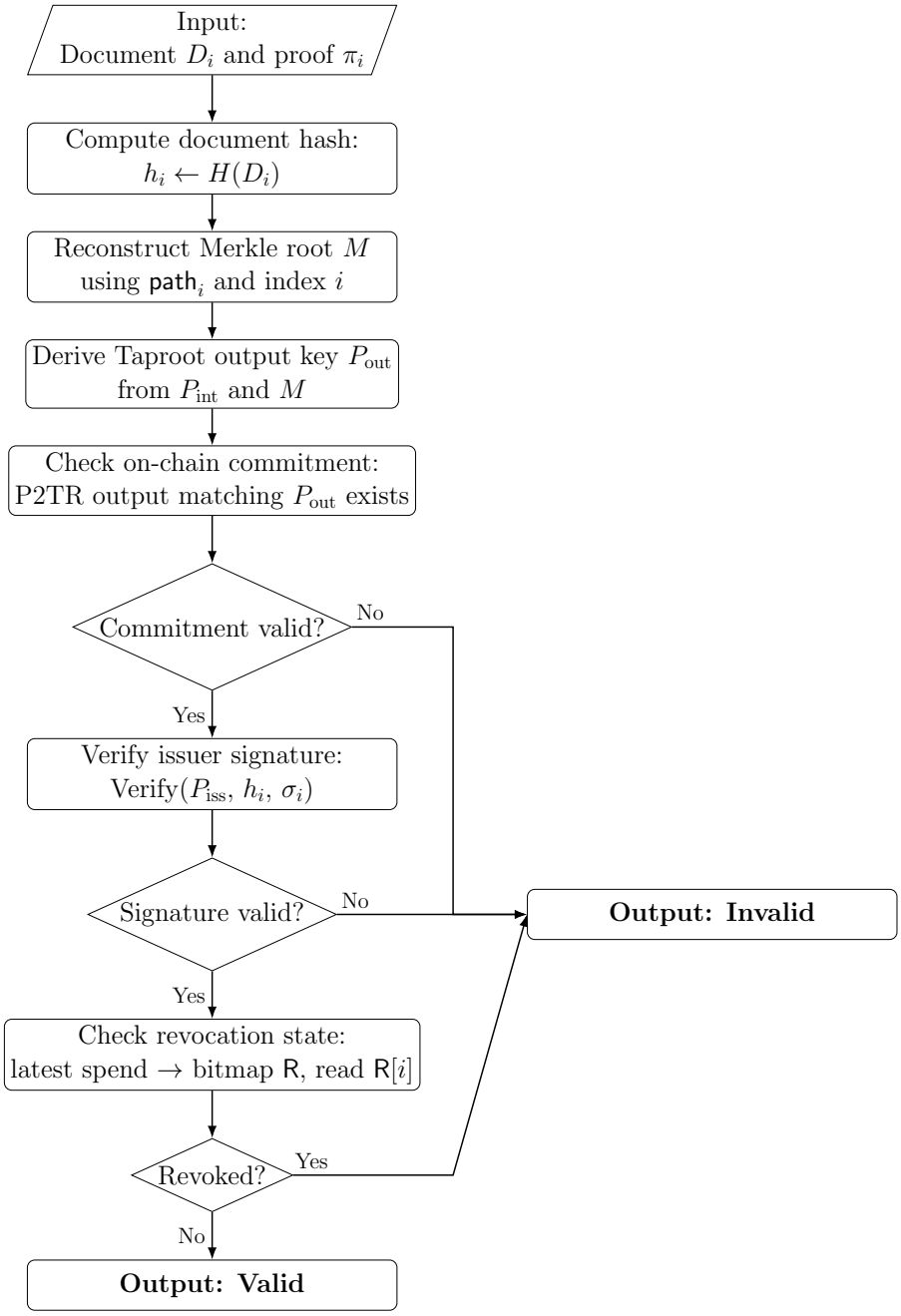


Figure 3: Verification workflow for a document and its proof-of-validation.

**Algorithm 2** Document Verification

**Require:** $D_i$, $\pi_i$
**Ensure:** Accept or Reject

```
h_i ← H(D_i)
Reconstruct ℓ_i and compute M using path_i
Derive P_out using Eq. (4)
Verify that transaction txid_0 contains an output paying to P_out
if UTXO not found then
    return Reject
end if
m_i ← H_sig(h_i ‖ i ‖ txid_0)
if Verify(P_iss, m_i, σ_i) = false then
    return Reject
end if
Retrieve latest revocation bitmap B_k
if B_k[i] = 1 then
    return Reject
end if
return Accept
```

Given a document $D_i$ and its corresponding proof-of-validation $\pi_i$, the verifier first computes the document hash and reconstructs the Merkle root using the authentication path contained in the proof. From the reconstructed root and the internal Taproot key, the verifier derives the Taproot output key $P_{\text{out}}$ and checks whether a corresponding P2TR output exists in the Bitcoin blockchain. If the commitment is valid, the verifier then checks the issuer's Schnorr signature bound to the document and the anchoring transaction. Finally, the verifier retrieves the most recent revocation bitmap and verifies that the corresponding index has not been marked as revoked. Only if all checks succeed is the document considered valid.

### *4.4. Revocation Mechanism*

Bitcoin immutability prevents modification of $T_0$. Revocation is implemented through transaction chaining. Let us $T_k$ denote the $k$-th descendant transaction that spends the previous P2TR output via the key path and recreates an equivalent output committing to the same Merkle root $M$. Each update transaction $T_k$ includes an additional `OP_RETURN` output encoding a

revocation bitmap:

$$B_k \in \{0,1\}^N, \tag{8}$$

where $B_k[i] = 1$ indicates revocation of document $D_i$.

The size of $B_k$ is $|B_k| = N$ bits, implying an on-chain data cost of $\lceil N/8 \rceil$ bytes per revocation update. The economic impact of this linear growth is evaluated in Section 6. The valid revocation state corresponds to the bitmap included in the most recent confirmed descendant transaction in the canonical blockchain.

### *4.5. Design Properties*

The proposed protocol satisfies the following security and functional properties:

1. Document Integrity. Only documents included in the original batch commitment can produce a valid verification proof. Any document outside the committed set cannot generate a valid Merkle authentication path consistent with the on-chain commitment.
2. Issuer Authenticity. The authenticity of issued documents is guaranteed through Schnorr signatures. Security follows from the EUF-CMA security of Schnorr signatures as specified in BIP340.
3. Revocation Soundness. Once a revocation update transaction is confirmed on-chain, any previously issued document marked as revoked can no longer be successfully verified as valid.
4. Commitment Privacy. Taproot ensures that unused script leaves remain hidden under the commitment structure. As a result, observers cannot distinguish whether a transaction encodes a simple payment or a commitment to a complex verification structure.

Formal security definitions and reduction arguments are presented in the following section.

## 5. Security Definitions and Proof Sketch

This section formalizes the security guarantees of the proposed construction under standard cryptographic assumptions. We define the adversarial model, specify formal security properties, and provide reduction-based proof sketches demonstrating that any successful adversary would imply a break of underlying primitives.

*5.1. Adversarial Model*

We consider a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ interacting with the system after the anchoring transaction $T_0$ has been confirmed. The adversary is granted the following capabilities:

- Access to all public blockchain data, including $T_0$ and its descendant transactions.
- Access to any document $D_i$ and its corresponding proof-of-validation $\pi_i$.
- The ability to generate arbitrary candidate documents $D^*$ and arbitrary forged proofs $\pi^*$.

The adversary does not control a majority of the global hash rate and therefore cannot rewrite confirmed blockchain history. We assume standard Nakamoto consensus security under the honest-majority assumption. Cryptographic primitives are modeled under standard security assumptions:

- The hash function $H(\cdot)$ is collision resistant.
- Schnorr signatures are EUF-CMA secure under BIP340.

**Definition 1 (Valid Document).** A document $D_i$ with proof $\pi_i = (\mathsf{txid}_0, i, P_{\text{iss}}, \sigma_i, \mathsf{path}_i)$ is considered valid if and only if:

1. $h_i = H(D_i)$ reconstructs a leaf $\ell_i$ consistent with the Merkle root $M$ using $\mathsf{path}_i$;
2. the derived $P_{\text{out}}$ corresponds to a confirmed UTXO in transaction $\mathsf{txid}_0$;
3. $\sigma_i$ verifies under $P_{\text{iss}}$ for message $m_i = H_{\text{sig}}(h_i \parallel i \parallel \mathsf{txid}_0)$;
4. the most recent revocation bitmap $B_k$ satisfies $B_k[i] = 0$.

**Definition 2 (Correctness).** If the issuer follows the protocol honestly and $D_i$ has not been revoked, then the probability that the verification algorithm rejects $(D_i, \pi_i)$ is negligible.

### *5.2. Security Assumptions*

The security of the proposed construction relies on several standard cryptographic and system-level assumptions commonly adopted in blockchain-based protocols.

First, the hash function $H(\cdot)$ instantiated with SHA-256 is assumed to be collision resistant. This property ensures that an adversary cannot construct two distinct inputs producing the same hash value, which guarantees the binding property of document commitments and the integrity of the Merkle tree structure.

Second, the Schnorr signature scheme defined in BIP340 is assumed to be EUF-CMA based on the hardness of the discrete logarithm problem in the secp256k1 elliptic curve group. Under this assumption, no adversary can generate a valid signature under the issuer's public key $P_{\text{iss}}$ without knowledge of the corresponding private key $x_{\text{iss}}$.

Finally, the security of the blockchain anchoring mechanism relies on the honest-majority assumption underlying Nakamoto consensus. Specifically, we assume that no adversary controls a majority of the global mining hash power, preventing modification of confirmed blockchain history and ensuring the immutability of the anchoring transaction $T_0$ and its descendant revocation transactions. Under these assumptions, the proposed construction achieves scalable, revocable, and authenticity-preserving document verification anchored to the Bitcoin blockchain.

### *5.3. Security Properties*

This analysis focuses on three fundamental properties: batch integrity, issuer authenticity, and revocation soundness. Additionally, we discuss the impact of the issuer's private key compromise and loss on the security guarantees of the system.

Integrity ensures that only documents originally included in the committed batch can be successfully verified.

**Definition 3 (Batch Integrity).** The construction satisfies batch integrity if no probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can produce a pair $(D^*, \pi^*)$ such that $D^*$ is not included in the original batch committed in transaction $T_0$, and the verification algorithm accepts $(D^*, \pi^*)$ with nonnegligible probability.

Intuitively, this property follows from the collision resistance of the hash function $H(\cdot)$ together with the binding property of the Merkle commitment rooted at $M$. Any successful forgery would require either constructing a

second preimage for $H(\cdot)$ or producing a valid Merkle authentication path inconsistent with the committed root.

Authenticity guarantees that accepted proofs originate from the legitimate issuer controlling the signing key associated with $P_{\text{iss}}$.

**Definition 4 (Authenticity).** The scheme satisfies issuer authenticity if no PPT adversary can produce a valid proof $\pi^*$ containing a signature $\sigma^*$ that verifies under $P_{\text{iss}}$ for a message that was not signed by the legitimate issuer, except with negligible probability.

This property follows from the existential unforgeability under chosen-message attack (EUF-CMA) security of Schnorr signatures as specified in BIP340.

Revocation soundness ensures that documents explicitly marked as revoked cannot be successfully verified.

**Definition 5 (Revocation Soundness).** The scheme satisfies revocation soundness if, once a revocation bitmap $B_k$ is confirmed on-chain with $B_k[i] = 1$, no PPT adversary can produce a proof $\pi_i$ that is accepted by the verification algorithm for document $D_i$.

This property follows from the requirement that the verification algorithm always evaluates the most recent confirmed revocation bitmap associated with the commitment chain originating from $T_0$.

Finally, we consider the implications of issuer private key compromise and key loss for the security of the system. If the issuer's Schnorr private key $x_{\text{iss}}$ is compromised after issuance, previously anchored documents remain cryptographically intact. Integrity depends only on the collision resistance of the committed hash values and the immutability of the anchoring transaction $T_0$. However, authenticity of future documents can no longer be guaranteed under the compromised key.

In contrast, if the private key is irreversibly lost, the issuer can no longer generate valid signatures for newly issued documents. Nevertheless, previously issued and anchored documents remain verifiable because verification requires only the public key $P_{\text{iss}}$ and the commitment recorded in $T_0$. Therefore, key loss affects only the ability to issue new documents, while previously anchored documents remain valid. This separation between commitment integrity and signing capability is a direct consequence of anchoring document commitments independently from the issuer's signing state.

*5.4. Reduction Arguments*

We now provide proof sketches showing that any successful adversary against the proposed construction would imply a break of the underlying cryptographic assumptions.

**Theorem 1 (Integrity).** If an adversary succeeds in the batch integrity game with non-negligible probability, then one can construct an algorithm that either finds a collision in the hash function $H(\cdot)$ or produces a Merkle authentication path inconsistent with the committed root $M$.

*Proof.* A successful adversary must produce a pair $(D^*, \pi^*)$ that is accepted by the verification algorithm while $D^*$ was not included in the original batch committed in $T_0$. Acceptance requires reconstruction of a Merkle authentication path leading to the root $M$. Since the Taproot output key $P_{\text{out}}$ is deterministically derived from $M$, the reconstructed root must equal the committed value. Therefore, the adversary must either construct two distinct inputs that hash to the same leaf value or produce an alternative authentication path yielding the same Merkle root. Both cases reduce to violating the collision resistance of $H(\cdot)$. □

**Theorem 2 (Authenticity).** If an adversary produces a valid proof containing a forged signature under the public key $P_{\text{iss}}$, then one can construct an algorithm that breaks the existential unforgeability under chosen-message attack (EUF-CMA) security of Schnorr signatures.

*Proof.* A proof $\pi_i$ is accepted only if the signature $\sigma_i$ verifies under $P_{\text{iss}}$ for the message $m_i = H_{\text{sig}}(h_i \parallel i \parallel \mathsf{txid}_0)$. Since the message binds the document hash, its batch position, and the anchoring transaction identifier, producing a valid signature $\sigma^*$ without knowledge of the secret key $x_{\text{iss}}$ constitutes a direct Schnorr forgery. Consequently, any adversary capable of producing such a proof can be used to break the EUF-CMA security of the Schnorr signature scheme defined in BIP340. □

**Theorem 3 (Revocation Soundness).** Assuming the immutability of confirmed blockchain history under Nakamoto consensus, revocation soundness holds.

*Proof.* Once a revocation bitmap $B_k$ is confirmed in a descendant transaction of $T_0$ with $B_k[i] = 1$, the verification algorithm deterministically rejects document $D_i$. A successful adversary would therefore need to either alter the confirmed revocation bitmap or rewrite blockchain history to remove the revocation transaction. Both scenarios contradict the honest-majority assumption underlying the security of Nakamoto consensus. □

### *5.5. Extended Attack Analysis*

Beyond the formal cryptographic reductions presented previously, we consider several additional attack vectors relevant to practical deployment of the proposed construction. These scenarios do not violate the underlying security assumptions but may arise in adversarial environments interacting with the protocol.

Replay attacks across distinct document batches are prevented through explicit binding of signatures to the anchoring transaction identifier. In particular, each verification message includes the value $\mathsf{txid}_0$, ensuring that a signature $\sigma_i$ produced for one batch commitment cannot be reused in another commitment context. Any attempt to reuse $\sigma_i$ across batches would therefore fail during verification because the reconstructed message would differ.

Selective disclosure attacks are mitigated through the use of Merkle authentication paths. A verifier receives only the authentication path $\mathsf{path}_i$ corresponding to the document being verified, which reveals no information about other documents contained in the batch. Consequently, the protocol enables per-document verification while preserving the privacy of unrelated batch elements.

Key compromise scenarios are also considered. If the issuer's private signing key $x_{\text{iss}}$ is compromised after document issuance, previously anchored documents remain cryptographically valid. Their integrity relies solely on the collision resistance of the committed hash values and the immutability of the anchoring transaction $T_0$. However, authenticity of future document issuances can no longer be guaranteed under the compromised key. This limitation is inherent to any public-key infrastructure and does not affect the security of already anchored commitments.

Finally, denial-of-service considerations are limited to infrastructure availability rather than protocol correctness. The verification procedure is stateless and requires only read-only access to blockchain data. As a result, denial-of-service attacks can only affect the availability of blockchain nodes

or network connectivity and cannot compromise the integrity of the anchored commitments.

## 6. Comparative Evaluation

This section evaluates the scalability and cost characteristics of the proposed construction as a function of the batch size $N$. The goal of this evaluation is to quantify how the protocol behaves when the number of committed documents increases. In particular, we analyze four metrics that directly affect the practicality of the system: the size of the anchoring transaction, the size of the verification proof, the computational cost of verification, and the cost of revocation updates.

All on-chain costs are expressed in virtual bytes (vB), which represent the canonical fee metric used in Bitcoin. Transaction fees are determined by the mempool fee rate (sat/vB), which fluctuates depending on network congestion. Because this parameter varies over time and is external to protocol design, reporting costs in satoshis or USD would introduce non-reproducible variability. For this reason, all transaction costs are expressed purely in vB, defined as

$$\mathrm{vB} = \frac{\text{weight units}}{4}, \tag{9}$$

following Bitcoin's weight accounting rules. Given a mempool fee rate $\rho$ (sat/vB), the resulting transaction fee would be

$$\mathrm{fee} = \rho \cdot \mathrm{vB}, \tag{10}$$

but $\rho$ is treated as an external parameter and is therefore not fixed in this analysis.

The evaluation focuses on four quantities that determine the scalability of the protocol:

- the issuance transaction size $\mathrm{vB}(T_0)$,
- the proof size $|\pi_i|$,
- the verification complexity $t_{\mathrm{ver}}(N)$,
- the revocation update transaction size $\mathrm{vB}(T_k)$.

The anchoring transaction $T_0$ embeds the Merkle root inside the Taproot output key. Because the Merkle root is a fixed-size value independent of the number of documents in the batch, the size of the issuance transaction does not depend on $N$. The transaction size can therefore be expressed as

$$\mathrm{vB}(T_0) = C_{\mathrm{iss}}, \tag{11}$$

where $C_{\mathrm{iss}}$ denotes the constant size of the transaction template containing a standard P2TR output. This property implies that the issuance cost remains constant regardless of the number of committed documents.

The proof size grows logarithmically with the batch size due to the Merkle authentication path. Let $d = \lceil \log_2 N \rceil$ denote the depth of the Merkle tree. Each proof contains $d$ sibling hashes (32 bytes each), one Schnorr signature (64 bytes), and several fixed-size metadata fields including the transaction identifier, index, and issuer public key. The resulting proof size can therefore be written as

$$|\pi_i| = C_{\mathrm{proof}} + 32\lceil \log_2 N \rceil. \tag{12}$$

Figure 4 illustrates the growth of the proof size as a function of the batch size $N$. The results confirm the logarithmic behavior predicted by Eq. (12), showing that proof size increases slowly even for large batches. This property enables scalable document verification while maintaining compact validation proofs.

Verification requires reconstructing the Merkle root and validating the issuer signature. Consequently, the computational cost consists of $\lceil \log_2 N \rceil$ hash evaluations and a single Schnorr signature verification. The verification time can therefore be modeled as

$$t_{\mathrm{ver}}(N) = a \cdot \lceil \log_2 N \rceil + b, \tag{13}$$

where the constants $a$ and $b$ depend on the implementation environment. Figure 5 shows the verification time as the batch size increases. The parameters $a = 0.02$ ms and $b = 0.10$ ms were obtained from experimental measurements of SHA-256 hashing and Schnorr signature verification on a standard workstation. The results confirm the logarithmic verification complexity predicted by Eq. (13), demonstrating that verification remains efficient even for large document batches.

Revocation is implemented through a bitmap published in the revocation transaction. If the batch contains $N$ documents, the bitmap requires

$$|B_k| = \left\lceil \frac{N}{8} \right\rceil \text{ bytes.} \tag{14}$$

The total size of the revocation transaction is therefore

$$\mathrm{vB}(T_k) = C_{\mathrm{rev}} + \left\lceil \frac{N}{8} \right\rceil, \tag{15}$$

where $C_{\mathrm{rev}}$ represents the constant overhead of the transaction template. For a fixed batch size $N$, subsequent revocation updates do not increase this value because each update republishes a bitmap of identical length. Figure 6 illustrates the revocation transaction size as the batch size increases. The results confirm the linear growth predicted by Eq. (15), since the revocation bitmap requires $\lceil N/8 \rceil$ bytes to encode the state of $N$ documents.

Table 6 summarizes the scalability properties of the proposed construction using the metrics described above. The table highlights how each component of the protocol behaves as the batch size increases. In particular, the issuance transaction size remains constant in vB, reflecting the $\mathcal{O}(1)$ anchoring cost achieved by committing the entire batch through a single Taproot output. In contrast, the proof size and verification complexity grow logarithmically with $N$, which is expected for Merkle-based constructions. Revocation updates grow linearly with the batch size because the bitmap encoding requires $\mathcal{O}(N)$ space.

The concrete instantiations for $2^{10}$, $2^{14}$, and $2^{16}$ documents demonstrate that proof growth remains moderate even for large batches, while the issuance cost remains constant and the verification procedure requires only a small number of hash operations together with a single signature verification.

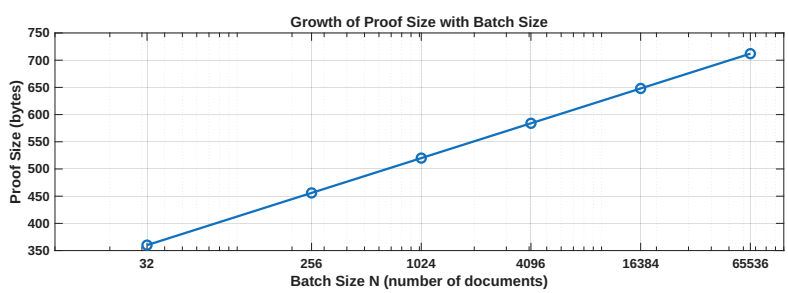


Figure 4: Proof size growth as a function of batch size $N$

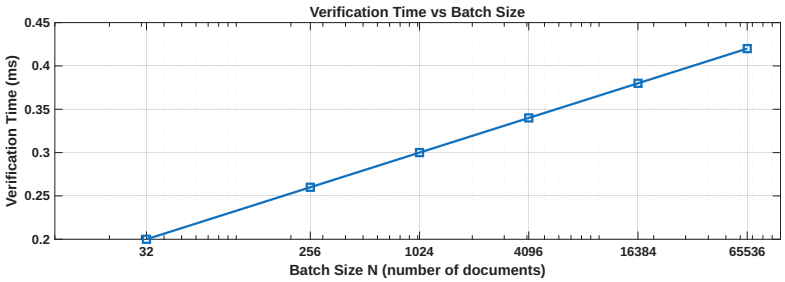


Figure 5: Verification time as a function of batch size $N$

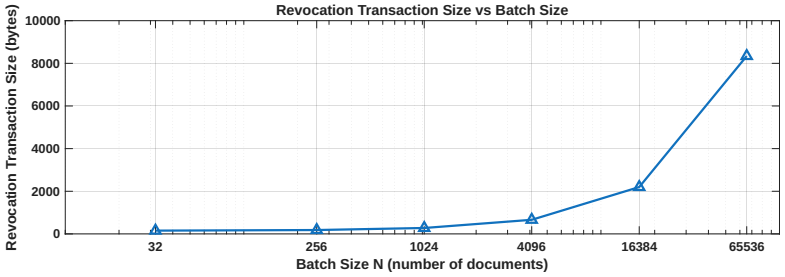


Figure 6: Revocation transaction size as a function of batch size $N$

Another comparison considers representative blockchain-based document anchoring approaches in order to contextualize the proposed construction. Traditional anchoring techniques typically embed document hashes directly in Bitcoin transactions using mechanisms such as `OP_RETURN`. While simple, this approach introduces a linear on-chain footprint because each document requires an individual commitment.

Merkle-based anchoring systems improve scalability by aggregating document hashes into a Merkle tree and anchoring only the root hash on-chain. This reduces the issuance cost to $\mathcal{O}(1)$ while preserving efficient per-document verification through logarithmic Merkle proofs. However, many existing systems do not natively support issuer authentication or explicit revocation mechanisms.

The proposed protocol preserves the scalability advantages of Merkle-based anchoring while integrating issuer authentication and revocation support through a Taproot-native construction. Table 7 provides a high-level comparison of representative Bitcoin document anchoring approaches, highlighting their scalability characteristics and support for authentication and revocation.

Table 6: Scalability of the proposed construction as a function of batch size $N$. On-chain costs are reported exclusively in virtual bytes (vB).

| **Batch size** $N$ | **Issuance size** vB($T_0$) | **Proof size** $\|\pi_i\|$ (bytes) | **Verification complexity** | **Revocation size** vB($T_k$) |
|---|---|---|---|---|
| $\mathcal{O}(N)$ | $\mathcal{O}(1)$ | $\mathcal{O}(\log N)$ | $\mathcal{O}(\log N)$ | $\mathcal{O}(N)$ |
| $2^{10}$ (1,024) | $C_{\text{iss}}$ | $C_{\text{proof}} + 320$ | $\sim$ 10 hash ops + 1 sig | $C_{\text{rev}} + 128$ |
| $2^{14}$ (16,384) | $C_{\text{iss}}$ | $C_{\text{proof}} + 448$ | $\sim$ 14 hash ops + 1 sig | $C_{\text{rev}} + 2{,}048$ |
| $2^{16}$ (65,536) | $C_{\text{iss}}$ | $C_{\text{proof}} + 512$ | $\sim$ 16 hash ops + 1 sig | $C_{\text{rev}} + 8{,}192$ |

Table 7: High-level comparison of representative Bitcoin document anchoring approaches.

| **Approach** | **Issuance Cost** | **Verification** | **Revocation** | **Authentication** |
|---|---|---|---|---|
| `OP_RETURN` Anchoring | $\mathcal{O}(N)$ | $\mathcal{O}(1)$ | No | No |
| Merkle Anchoring (e.g., OpenTimestamps) | $\mathcal{O}(1)$ | $\mathcal{O}(\log N)$ | No | No |
| Proposed Protocol | $\mathcal{O}(1)$ | $\mathcal{O}(\log N)$ | Yes | Yes |

This section discusses the broader implications of the proposed protocol, including its security and privacy guarantees, operational trade-offs, and deployment considerations. The analysis also highlights practical limitations and outlines directions for future research.

### *6.1. Security Implications*

The security of the proposed construction relies on three standard assumptions: (i) collision resistance of the hash function $H(\cdot)$ (e.g., SHA-256), (ii) existential unforgeability under chosen-message attacks (EUF-CMA) of Schnorr signatures defined in BIP340, and (iii) the honest-majority security of Nakamoto consensus. Under these assumptions, the protocol guarantees batch integrity, issuer authenticity, and revocation soundness as defined in Section 5. Once the anchoring transaction $T_0$ is confirmed, document inclusion becomes immutable unless an adversary can either violate the collision resistance of $H(\cdot)$ or rewrite confirmed blockchain history.

Key management for the issuer keys $P_{\text{int}}$ and $x_{\text{iss}}$ remains orthogonal to the protocol design. However, the linearity of Schnorr signatures enables straightforward integration of threshold or multisignature constructions without modifying the verification algorithm. This property is particularly useful for institutional deployments where distributed authorization or multi-party

approval is required.

### *6.2. Privacy Considerations*

The protocol provides selective disclosure at the batch level through the use of Merkle authentication paths. During verification, only the authentication path corresponding to the verified document is revealed, while other leaves in the batch remain hidden. As a result, verification does not expose unrelated documents or their contents.

Nevertheless, all documents anchored in the same transaction remain publicly linkable through the transaction identifier $\mathsf{txid}_0$. Consequently, the protocol preserves confidentiality of document contents and unrelated batch elements, but does not provide unlinkability between documents issued within the same batch.

### *6.3. Scalability Trade-offs*

The primary scalability trade-off arises from the revocation bitmap encoding used by the protocol. As analyzed in Section 6, the size of revocation transactions grows linearly with the batch size $N$, since the bitmap requires $\lceil N/8 \rceil$ bytes. Consequently, the protocol exhibits an asymmetric scalability profile: issuance cost remains constant and verification complexity grows logarithmically, while revocation updates introduce linear on-chain growth. For a fixed batch size $N$, subsequent revocation updates do not increase the bitmap length, meaning that each additional revocation update incurs constant marginal cost once the batch size is fixed.

In practice, relay policy limits on `OP_RETURN` payload size impose an upper bound on feasible batch sizes when revocation support is required. These limits must therefore be considered when selecting batch sizes for operational deployments. Alternative revocation encodings could further improve asymptotic scalability. For example, Merkle-based accumulators or sparse Merkle trees could replace the bitmap representation while preserving logarithmic verification complexity. Such approaches represent an interesting direction for future work.

### *6.4. Deployment Considerations*

The proposed construction is fully compatible with existing Bitcoin infrastructure and requires no modifications to the underlying protocol. Verification is entirely read-only and does not require interaction with the issuer once the proof-of-validation has been distributed.

Operational deployments should nevertheless consider several organizational factors, including key rotation policies, long-term archival storage of proofs, and the selection of batch sizes relative to expected revocation frequency. Careful coordination between issuance policies and revocation requirements is necessary to maintain long-term operational efficiency.

Future work includes empirical benchmarking of issuance and revocation costs under real mainnet conditions, exploration of alternative revocation encodings such as Merkle accumulators, and adaptive batch segmentation strategies that balance issuance efficiency with revocation overhead.

### *6.5. Institutional Key Governance*

To mitigate risks associated with single-key control, the issuer key $P_{\text{iss}}$ can be instantiated as an aggregated MuSig2 public key derived from multiple institutional actors. For example, a university deployment may construct a three-party aggregated key composed of the Academic Director of the department, the Head of Information Systems, and the Dean of the faculty. Using MuSig2 key aggregation, these actors jointly generate a single aggregated public key $P_{\text{agg}}$ that replaces $P_{\text{iss}}$ in the protocol.

Document signatures are then generated through interactive threshold signing while keeping individual private key shares confidential. This approach eliminates single points of failure, prevents unilateral document issuance, and improves institutional accountability through distributed authorization. Importantly, MuSig2 preserves the single public key abstraction of Schnorr signatures. As a result, verifiers continue to validate a single signature under $P_{\text{agg}}$, meaning that the verification procedure remains unchanged while operational security is strengthened. This design prevents unilateral document issuance and reduces the risk associated with single-key compromise in institutional deployments.

### *6.6. Performance Outlook*

Overall, the protocol achieves constant-size issuance cost, logarithmic verification complexity $O(\log N)$, and linear revocation update size $O(N)$. This asymmetric scaling profile shifts long-term operational costs toward revocation management while preserving efficient verification even for very large document batches.

## 7. Conclusion and Future Work

This paper presents a Taproot-based document verification protocol that enables scalable batch commitments, issuer authenticity guarantees, and explicit revocation support using only standardized Bitcoin primitives. Unlike prior Bitcoin-based anchoring approaches that focus solely on integrity, the proposed construction integrates batch aggregation, signature binding, and lifecycle state management within a unified Taproot-native design. The protocol achieves constant-size issuance cost, logarithmic verification complexity, and linear revocation update cost with respect to batch size. By leveraging Schnorr signatures and Merkleized script commitments, the construction preserves privacy of non-related batch elements and avoids on-chain disclosure of document data.

Revocation is implemented through publicly auditable state transitions without invalidating prior commitments. Importantly, the results demonstrate that expressive smart contract execution is not required to implement scalable and revocable certification systems at the Bitcoin base layer. Taproot provides sufficient cryptographic structure to support advanced non-monetary applications while preserving Bitcoin's minimal execution model.

Future research directions include alternative revocation encodings with improved asymptotic behavior, privacy-enhancing verification mechanisms, and long-term archival strategies for proof persistence. Overall, the findings indicate that Taproot significantly expands the design space for decentralized verification systems on Bitcoin, enabling practical, privacy-preserving certification architectures that operate without trusted intermediaries. Future work may also explore privacy-enhancing extensions such as zero-knowledge proofs capable of hiding Merkle path positions or issuer metadata. Such approaches not only improve unlinkability properties, they would also introduce additional computational overhead and implementation complexity.

## Acknowledgment

The author would like to thank King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia, for supporting this research. Authors would like to acknowledge the support provided by the Interdisciplinary Research Center for Intelligent Secure Systems (IRC-ISS) and the Deanship of Scientific Research at King Fahd University of Petroleum & Minerals. This project is funded by IRC-ISS Grants #INSS2502 and #INSS2503.

## References


[1] R. Xie, W. Zhu, Y. Zhao, Q. Han, Z. Liu, J.-W. Kim, J. Choi, Ethereum blockchain decentralized certificate management system based on smart contract, IEEE Access 8 (2020) 90618–90632. doi:10.1109/ACCESS.2020.2993881.

[2] A. Shende, M. Mullapudi, N. Challa, Enhancing document verification systems: A review of techniques, challenges, and practical implementations, International Journal of Computer Engineering & Technology 15 (2024) 16–25.

[3] S. Nakamoto, Bitcoin: A peer-to-peer electronic cash system, `https://bitcoin.org/bitcoin.pdf` (2008).

[4] V. L. Lemieux, Trusting records: is blockchain technology the answer?, Records Management Journal 26 (2) (2016) 110–139. doi:10.1108/RMJ-12-2015-0042.

[5] P. Wuille, J. Nick, T. Ruffing, BIP 340: Schnorr Signatures for secp256k1, Bitcoin Improvement Proposals, accessed: 2025-11-20 (2020).
URL `https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki`

[6] P. Wuille, J. Nick, A. Towns, BIP 341: Taproot: SegWit version 1 spending rules, Bitcoin Improvement Proposals, accessed: 2025-11-20 (2020).
URL `https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki`

[7] Y. Gao, H. Nobuhara, A decentralized trusted timestamping based on blockchains, IEEJ Journal of Industry Applications 6 (4) (2017) 252–257. doi:10.1541/ieejjia.6.252.

[8] M. Bartoletti, B. Bellomy, L. Pompianu, A journey into Bitcoin metadata, Journal of Grid Computing 17 (2019) 3–22. doi:10.1007/s10723-019-09473-3.

[9] E. Strehle, F. Steinmetz, Dominating OP returns: The impact of omni and veriblock on bitcoin, Journal of Grid Computing 18 (2020) 575–590. doi:10.1007/s10723-020-09537-9.

[10] M. Kedziora, D. Pieprzka, I. Jozwiak, Y. Liu, H. Song, Analysis of segregated witness implementation for increasing efficiency and security of the bitcoin cryptocurrency, Journal of Information and Telecommunication 7 (1) (2023) 44–55, published online: 26 Sep 2022. doi:10.1080/24751839.2022.2122301.

[11] Blockcerts Project, Blockcerts: Open standard for blockchain credentials, `https://www.blockcerts.org/` (2024).

[12] P. Todd, Opentimestamps, `https://opentimestamps.org/` (2024).

[13] V. Buterin, Ethereum: A next-generation smart contract and decentralized application platform, `https://ethereum.org/en/whitepaper/` (2014).

[14] Z. Wang, J. Lin, Q. Cai, Q. Wang, J. Jing, D. Zha, Blockchain-based certificate transparency and revocation transparency, in: A. Zohar, I. Eyal, V. Teague, J. Clark, A. Bracciali, F. Pintore, M. Sala (Eds.), Financial Cryptography and Data Security: FC 2018 International Workshops, BITCOIN, VOTING, and WTSC, Revised Selected Papers, Vol. 10958 of Lecture Notes in Computer Science, Springer, 2019, pp. 144–164.

[15] Y. Li, Y. Yu, C. Lou, N. Guizani, L. Wang, Decentralized public key infrastructures atop blockchain, IEEE Network (2020). doi:10.1109/MNET.011.2000085.

[16] D. Pennino, M. Pizzonia, A. Vitaletti, M. Zecchini, Efficient certification of endpoint control on blockchain, IEEE Access 9 (2021) 131901–131915. doi:10.1109/ACCESS.2021.3115343.

[17] T. Palmisano, V. N. Convertini, L. Sarcinella, L. Gabriele, M. Bonifazi, Notarization and anti-plagiarism: A new blockchain approach, Applied Sciences 12 (1) (2022) 243. doi:10.3390/app12010243.

[18] A. Mishra, S. Mehta, B. Oza, S. Kumar, H. Kasturiwale, Blockchain-based decentralized document verification and its applications, Journal of Information Systems Engineering and Management 10 (10s) (2025).

[19] F. Härer, H.-G. Fill, Decentralized attestation and distribution of information using blockchains and multi-protocol storage, IEEE Access 10 (2022) 20017–20032. doi:10.1109/ACCESS.2022.3150356.

[20] R. Q. Castro, M. Au-Yong-Oliveira, Blockchain and higher education diplomas, Eur. J. Investig. Health Psychol. Educ. 11 (1) (2021) 154–167. doi:10.3390/ejihpe11010013.

[21] A. Rustemi, F. Dalipi, V. Atanasovski, A. Risteski, A systematic literature review on blockchain-based systems for academic certificate verification, IEEE Access 11 (2023) 66498–66518. doi:10.1109/ACCESS.2023.3289598.

[22] Šeila Bećirović Ramić, E. Cogo, I. Prazina, E. Cogo, M. Turkanović, R. T. Mulahasanović, S. Mrdović, Selective disclosure in digital credentials: A review, ICT Express 10 (2024) 916–934. doi:10.1016/j.icte.2024.05.011.

[23] T. Zhang, Y. Wang, B. Gong, J. Xu, J. Wu, C. Wan, Privacy protection during the issuance and revocation of verifiable credentials in self-sovereign identity, Concurrency and Computation: Practice and Experience 37 (9) (2025) e70084. doi:10.1002/cpe.70084.